

What's On My Way — Product Specification

Status: Draft v2 — incorporates Draft Specification, and two rounds of design discussion.

1. Product Overview

An iPhone/iPad app where a user plans a route (point A → point B), picks a search radius off that route (1, 5, 10, 25, or 50 miles), and sees points of interest (POIs) within that corridor. POI data comes from purchasable **Packs** — focused databases (e.g., all Waffle Houses, or unusual roadside attractions) — each with its own pack-specific filters (e.g., 24-hour, delivery, dog-friendly). **State Packs** are a derived pack type containing every POI (across all types) within a given state.

Confirmed product decisions

- **Purchases:** Apple In-App Purchase (StoreKit). Each pack is a non-consumable IAP product; ownership is tied to the Apple account used for purchase, including Family Sharing members.
- **Data locality:** Offline-first. Purchased pack data downloads to the device; route search runs locally without a network call.
- **"Along the route" filtering for state packs:** No special geographic boundary logic needed — a state pack only contains POIs from that state, so the radius filter naturally limits results to the portion of the route passing through it.
- **Pack authorship:** Initially, packs are built, updated, and published by the product owner only. Third parties may submit data for inclusion, but import/curation is manually managed.
- **Additional data:** Every POI has exactly one freeform `additional_info` text field (not searchable, not filterable), shown only in the POI detail view, only when non-empty.
- **Routing:** Apple MapKit (`MKDirections`).
- **Location permissions: No device location for v1.** Start and end points are always typed/selected manually — no "use current location" button, no location permission prompt, simplest possible privacy label. A convenience location button is an explicit future option once usage data justifies the added permission/review overhead.
- **Local spatial search:** Bounding-box pre-filter (indexed) + linear haversine on survivors, benchmarked in Phase 1; full R-tree spatial index deferred until the benchmark shows it's needed.
- **Geocoding:** Smarty (US address standardization + geocoding in one call), chosen because its terms permit permanent storage of results (unlike Google's Geocoding API) and because standardization directly addresses messy third-party address data.
- **Platform scope:** iPhone and iPad only. No Android/web end-user client — web is admin/internal only.

2. Technology Stack

Backend (VPS)

Layer	Technology
Runtime	Node.js + TypeScript
Framework	Fastify
ORM	Drizzle ORM
Database	PostgreSQL 16 (dedicated container)
Auth	Sign in with Apple → backend session (JWT access + rotating refresh tokens)
Payments	Apple StoreKit 2 (client) + App Store Server API (server-side receipt/transaction verification) + App Store Server Notifications V2 (refund/revoke webhook)
Geocoding	Smarty (US address standardization + geocoding)
Container	Docker, bound to <code>127.0.0.1:<port></code> , proxied by nginx (same pattern as existing VPS services)

iOS App

Layer	Technology
Framework	Expo (React Native) + TypeScript
Navigation	Expo Router
UI	NativeWind (Tailwind for React Native) + small set of custom components
Routing/Directions	Apple MapKit (<code>MKDirections</code>)
Local storage	SQLite (<code>expo-sqlite</code>) — installed pack data, offline search
Downloads	<code>expo-file-system</code> <code>createDownloadResumable</code> (resumable, Range-based)
Purchases	<code>react-native-iap</code> or Expo's StoreKit 2 bindings
Auth Storage	<code>expo-secure-store</code>
Remote State	TanStack Query
Validation	Zod

Web Interface (Admin: pack authoring, master DB, imports)

Layer	Technology
Framework	Next.js + TypeScript (static export)
UI	shadcn/ui + Tailwind CSS
State	TanStack Query
Validation	Zod

3. VPS Infrastructure

Follows the existing pattern used on `rycsprojects` (Ubuntu 24.04, Docker + nginx + certbot + UFW + fail2ban).

Component	Detail
API container	<code>wowm-api</code> (Fastify + Drizzle), bound to <code>127.0.0.1:3003</code>
DB container	<code>wowm-db</code> (<code>postgres:16-alpine</code>), internal only, dedicated volume <code>wowm_db-data</code>
Domain	<code>wowm.rycsprojects.com</code> (or final product domain) — nginx reverse proxy, HTTPS via certbot, HSTS + security headers per existing global config
Admin web	Static export under the same domain or a subdomain, same static-hosting pattern as other sites
Pack file storage	Served only via authenticated API endpoint (entitlement-gated), with <code>Range</code> support for resumable client downloads
Rate limiting	Add <code>wowm</code> API locations to existing <code>zone=api</code> (20r/s) nginx rate limit
Backups	Extend <code>vps-backup.sh</code> : <code>pg_dump</code> for <code>wowm-db</code>
Monitoring	Extend <code>server-monitor.sh</code> Docker check to include <code>wowm-api</code> / <code>wowm-db</code>
Secrets	Apple App Store Server API key (<code>.p8</code>), Sign in with Apple config, JWT signing secret, Smarty API key — in <code>wowm/infra/.env</code>

4. SQL Data Model (PostgreSQL — backend / master database)

```
-- Users (app accounts, tied to Sign in with Apple)
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  apple_sub TEXT NOT NULL UNIQUE,          -- 'sub' claim from Sign in with Apple
  email TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- POI Types (categorizes POIs; a standard pack maps to one poi_type)
CREATE TABLE poi_types (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug TEXT NOT NULL UNIQUE,
  name TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Packs (purchasable products; standard or state)
CREATE TABLE packs (
```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
slug TEXT NOT NULL UNIQUE,
name TEXT NOT NULL,
description TEXT,
pack_type TEXT NOT NULL CHECK (pack_type IN ('standard', 'state')),
poi_type_id UUID REFERENCES poi_types(id), -- NULL for state packs (mixed types)
state_code CHAR(2), -- set only for state packs
apple_product_id TEXT NOT NULL UNIQUE, -- StoreKit product identifier
price_tier TEXT,
status TEXT NOT NULL DEFAULT 'draft' CHECK (status IN ('draft', 'published', 'deprecated')),
current_version INT NOT NULL DEFAULT 0,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Pack framework: pack-specific additional data fields
CREATE TABLE pack_field_definitions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  pack_id UUID NOT NULL REFERENCES packs(id) ON DELETE CASCADE,
  field_key TEXT NOT NULL,
  label TEXT NOT NULL,
  data_type TEXT NOT NULL CHECK (data_type IN ('text', 'number', 'boolean', 'url', 'phone', 'enum')),
  enum_options JSONB,
  is_required BOOLEAN NOT NULL DEFAULT false,
  sort_order INT NOT NULL DEFAULT 0,
  UNIQUE (pack_id, field_key)
);

-- Pack framework: pack-specific filters
CREATE TABLE pack_filter_definitions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  pack_id UUID NOT NULL REFERENCES packs(id) ON DELETE CASCADE,
  filter_key TEXT NOT NULL,
  label TEXT NOT NULL,
  filter_type TEXT NOT NULL CHECK (filter_type IN ('boolean', 'single-select', 'multi-select')),
  options JSONB,
  sort_order INT NOT NULL DEFAULT 0,
  UNIQUE (pack_id, filter_key)
);

-- Master POI table (source of truth for all points of interest)
CREATE TABLE pois (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  poi_type_id UUID NOT NULL REFERENCES poi_types(id),
  name TEXT NOT NULL,
  address_street TEXT NOT NULL,
  address_city TEXT NOT NULL,
  address_state CHAR(2) NOT NULL,
  address_zip TEXT NOT NULL,
  phone TEXT,
  website TEXT,
  additional_info TEXT, -- single freeform field, detail view only
  latitude DOUBLE PRECISION,
  longitude DOUBLE PRECISION,
  geocode_status TEXT NOT NULL DEFAULT 'pending' CHECK (geocode_status IN
('pending', 'ok', 'low_confidence', 'failed')),
  geocode_confidence DOUBLE PRECISION, -- provider confidence score, 0-1
  geocode_candidates JSONB, -- candidate matches when ADDRESS_AMBIGUOUS
  custom_fields JSONB NOT NULL DEFAULT '{}', -- keyed by pack_field_definitions.field_key
  filter_values JSONB NOT NULL DEFAULT '{}', -- keyed by pack_filter_definitions.filter_key
  status TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'flagged', 'archived')),

```

```

    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX pois_poi_type_idx ON pois (poi_type_id);
CREATE INDEX pois_state_idx ON pois (address_state);
CREATE INDEX pois_lat_lng_idx ON pois (latitude, longitude); -- bounding-box pre-filter; free
path to future "nearby" admin tooling without PostGIS

-- Pack <-> POI membership snapshot (rebuilt whenever a pack is generated/updated)
CREATE TABLE pack_pois (
    pack_id UUID NOT NULL REFERENCES packs(id) ON DELETE CASCADE,
    poi_id UUID NOT NULL REFERENCES pois(id) ON DELETE CASCADE,
    PRIMARY KEY (pack_id, poi_id)
);

-- Versioned, downloadable pack releases
CREATE TABLE pack_versions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    pack_id UUID NOT NULL REFERENCES packs(id) ON DELETE CASCADE,
    version INT NOT NULL,
    format_version INT NOT NULL DEFAULT 1, -- on-device pack FILE format; app refuses formats it
doesn't understand
    file_url TEXT NOT NULL,
    checksum TEXT NOT NULL,
    poi_count INT NOT NULL,
    changelog TEXT,
    released_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (pack_id, version)
);

-- Entitlements: which Apple account owns which pack
CREATE TABLE entitlements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    pack_id UUID NOT NULL REFERENCES packs(id),
    apple_original_transaction_id TEXT NOT NULL,
    apple_transaction_id TEXT NOT NULL,
    environment TEXT NOT NULL DEFAULT 'production' CHECK (environment IN ('sandbox', 'production')),
    purchased_at TIMESTAMPTZ NOT NULL,
    revoked_at TIMESTAMPTZ, -- set via App Store Server Notifications V2
    (refund/revoke)
    UNIQUE (user_id, pack_id)
);

-- Data import batches (admin tooling)
CREATE TABLE imports (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    poi_type_id UUID NOT NULL REFERENCES poi_types(id),
    filename TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'processing' CHECK (status IN ('processing', 'completed', 'failed')),
    total_rows INT NOT NULL DEFAULT 0,
    flagged_rows INT NOT NULL DEFAULT 0,
    error_message TEXT, -- set when import hard-stops (missing required field)
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    completed_at TIMESTAMPTZ
);

CREATE TABLE import_row_errors (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    import_id UUID NOT NULL REFERENCES imports(id) ON DELETE CASCADE,
    row_number INT NOT NULL,

```

```

error_code TEXT NOT NULL,          -- see Error Taxonomy (§15)
field TEXT,
message TEXT NOT NULL
);

-- In-app feedback / data-correction submissions
CREATE TABLE feedback_submissions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),    -- nullable, anonymous submissions allowed
  pack_slug TEXT,
  poi_name TEXT,
  poi_address TEXT,
  message TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'open' CHECK (status IN ('open','resolved')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

5. On-Device Data Model (SQLite — iOS local pack storage)

Denormalized for fast local reads; repopulated on pack install/update from the downloaded pack file.

```

-- Schema version tracked via PRAGMA user_version; migrations run in order on launch
-- before any pack install/update path is allowed to proceed.

CREATE TABLE installed_packs (
  pack_id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  pack_type TEXT NOT NULL,
  version INTEGER NOT NULL,
  format_version INTEGER NOT NULL,
  downloaded_at TEXT NOT NULL,
  is_active INTEGER NOT NULL DEFAULT 1  -- toggled by multi-pack selection UI
);

CREATE TABLE pack_field_defs (
  pack_id TEXT NOT NULL,
  field_key TEXT NOT NULL,
  label TEXT NOT NULL,
  data_type TEXT NOT NULL,
  enum_options TEXT,          -- JSON string
  sort_order INTEGER NOT NULL,
  PRIMARY KEY (pack_id, field_key)
);

CREATE TABLE pack_filter_defs (
  pack_id TEXT NOT NULL,
  filter_key TEXT NOT NULL,
  label TEXT NOT NULL,
  filter_type TEXT NOT NULL,
  options TEXT,              -- JSON string
  sort_order INTEGER NOT NULL,
  PRIMARY KEY (pack_id, filter_key)
);

CREATE TABLE pois (
  id TEXT PRIMARY KEY,

```

```

pack_id TEXT NOT NULL,
poi_type_slug TEXT NOT NULL,
name TEXT NOT NULL,
address_street TEXT NOT NULL,
address_city TEXT NOT NULL,
address_state TEXT NOT NULL,
address_zip TEXT NOT NULL,
phone TEXT,
website TEXT,
additional_info TEXT,
latitude REAL NOT NULL,
longitude REAL NOT NULL,
custom_fields TEXT NOT NULL DEFAULT '{}', -- JSON string
filter_values TEXT NOT NULL DEFAULT '{}', -- JSON string
);
CREATE INDEX pois_pack_idx ON pois (pack_id);
CREATE INDEX pois_lat_lng_idx ON pois (latitude, longitude); -- bounding-box pre-filter
-- Revisit: SQLite R-tree spatial index if Phase 1 benchmark shows the bounding-box
-- pre-filter + haversine isn't enough at real pack sizes / multi-pack combinations.

```

6. Drizzle ORM Schema (backend)

```

import {
  pgTable, uuid, text, char, boolean, integer,
  jsonb, timestamp, doublePrecision, unique, primaryKey, index,
} from 'drizzle-orm/pg-core';

export const users = pgTable('users', {
  id: uuid('id').primaryKey().defaultRandom(),
  appleSub: text('apple_sub').notNull().unique(),
  email: text('email'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).notNull().defaultNow(),
});

export const poiTypes = pgTable('poi_types', {
  id: uuid('id').primaryKey().defaultRandom(),
  slug: text('slug').notNull().unique(),
  name: text('name').notNull(),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const packs = pgTable('packs', {
  id: uuid('id').primaryKey().defaultRandom(),
  slug: text('slug').notNull().unique(),
  name: text('name').notNull(),
  description: text('description'),
  packType: text('pack_type', { enum: ['standard', 'state'] }).notNull(),
  poiTypeId: uuid('poi_type_id').references(() => poiTypes.id),
  stateCode: char('state_code', { length: 2 }),
  appleProductId: text('apple_product_id').notNull().unique(),
  priceTier: text('price_tier'),
  status: text('status', { enum: ['draft', 'published', 'deprecated']
}).notNull().default('draft'),
  currentVersion: integer('current_version').notNull().default(0),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).notNull().defaultNow(),
});

```

```

});

export const packFieldDefinitions = pgTable('pack_field_definitions', {
  id: uuid('id').primaryKey().defaultRandom(),
  packId: uuid('pack_id').notNull().references(() => packs.id, { onDelete: 'cascade' }),
  fieldKey: text('field_key').notNull(),
  label: text('label').notNull(),
  dataType: text('data_type', { enum: ['text', 'number', 'boolean', 'url', 'phone', 'enum']
}).notNull(),
  enumOptions: jsonb('enum_options'),
  isRequired: boolean('is_required').notNull().default(false),
  sortOrder: integer('sort_order').notNull().default(0),
}, (t) => ({
  uniq: unique().on(t.packId, t.fieldKey),
}));

export const packFilterDefinitions = pgTable('pack_filter_definitions', {
  id: uuid('id').primaryKey().defaultRandom(),
  packId: uuid('pack_id').notNull().references(() => packs.id, { onDelete: 'cascade' }),
  filterKey: text('filter_key').notNull(),
  label: text('label').notNull(),
  filterType: text('filter_type', { enum: ['boolean', 'single-select', 'multi-select']
}).notNull(),
  options: jsonb('options'),
  sortOrder: integer('sort_order').notNull().default(0),
}, (t) => ({
  uniq: unique().on(t.packId, t.filterKey),
}));

export const pois = pgTable('pois', {
  id: uuid('id').primaryKey().defaultRandom(),
  poiTypeId: uuid('poi_type_id').notNull().references(() => poiTypes.id),
  name: text('name').notNull(),
  addressStreet: text('address_street').notNull(),
  addressCity: text('address_city').notNull(),
  addressState: char('address_state', { length: 2 }).notNull(),
  addressZip: text('address_zip').notNull(),
  phone: text('phone'),
  website: text('website'),
  additionalInfo: text('additional_info'),
  latitude: doublePrecision('latitude'),
  longitude: doublePrecision('longitude'),
  geocodeStatus: text('geocode_status', { enum: ['pending', 'ok', 'low_confidence', 'failed']
}).notNull().default('pending'),
  geocodeConfidence: doublePrecision('geocode_confidence'),
  geocodeCandidates: jsonb('geocode_candidates'),
  customFields: jsonb('custom_fields').notNull().default({}),
  filterValues: jsonb('filter_values').notNull().default({}),
  status: text('status', { enum: ['active', 'flagged', 'archived'] }).notNull().default('active'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true }).notNull().defaultNow(),
}, (t) => ({
  poiTypeIdx: index('pois_poi_type_idx').on(t.poiTypeId),
  stateIdx: index('pois_state_idx').on(t.addressState),
  latLngIdx: index('pois_lat_lng_idx').on(t.latitude, t.longitude),
}));

export const packPois = pgTable('pack_pois', {
  packId: uuid('pack_id').notNull().references(() => packs.id, { onDelete: 'cascade' }),
  poiId: uuid('poi_id').notNull().references(() => pois.id, { onDelete: 'cascade' }),
}, (t) => ({

```

```

    pk: primaryKey({ columns: [t.packId, t.poiId] }),
  }));

export const packVersions = pgTable('pack_versions', {
  id: uuid('id').primaryKey().defaultRandom(),
  packId: uuid('pack_id').notNull().references(() => packs.id, { onDelete: 'cascade' }),
  version: integer('version').notNull(),
  formatVersion: integer('format_version').notNull().default(1),
  fileUrl: text('file_url').notNull(),
  checksum: text('checksum').notNull(),
  poiCount: integer('poi_count').notNull(),
  changelog: text('changelog'),
  releasedAt: timestamp('released_at', { withTimezone: true }).notNull().defaultNow(),
}, (t) => ({
  uniq: unique().on(t.packId, t.version),
}));

export const entitlements = pgTable('entitlements', {
  id: uuid('id').primaryKey().defaultRandom(),
  userId: uuid('user_id').notNull().references(() => users.id, { onDelete: 'cascade' }),
  packId: uuid('pack_id').notNull().references(() => packs.id),
  appleOriginalTransactionId: text('apple_original_transaction_id').notNull(),
  appleTransactionId: text('apple_transaction_id').notNull(),
  environment: text('environment', { enum: ['sandbox', 'production']
}).notNull().default('production'),
  purchasedAt: timestamp('purchased_at', { withTimezone: true }).notNull(),
  revokedAt: timestamp('revoked_at', { withTimezone: true }),
}, (t) => ({
  uniq: unique().on(t.userId, t.packId),
}));

export const imports = pgTable('imports', {
  id: uuid('id').primaryKey().defaultRandom(),
  poiTypeId: uuid('poi_type_id').notNull().references(() => poiTypes.id),
  filename: text('filename').notNull(),
  status: text('status', { enum: ['processing', 'completed', 'failed']
}).notNull().default('processing'),
  totalRows: integer('total_rows').notNull().default(0),
  flaggedRows: integer('flagged_rows').notNull().default(0),
  errorMessage: text('error_message'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  completedAt: timestamp('completed_at', { withTimezone: true }),
});

export const importRowErrors = pgTable('import_row_errors', {
  id: uuid('id').primaryKey().defaultRandom(),
  importId: uuid('import_id').notNull().references(() => imports.id, { onDelete: 'cascade' }),
  rowNumber: integer('row_number').notNull(),
  errorCode: text('error_code').notNull(),
  field: text('field'),
  message: text('message').notNull(),
});

export const feedbackSubmissions = pgTable('feedback_submissions', {
  id: uuid('id').primaryKey().defaultRandom(),
  userId: uuid('user_id').references(() => users.id),
  packSlug: text('pack_slug'),
  poiName: text('poi_name'),
  poiAddress: text('poi_address'),
  message: text('message').notNull(),
  status: text('status', { enum: ['open', 'resolved'] }).notNull().default('open'),

```

```
createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});
```

7. API Endpoints

Auth

Method	Path	Auth	Description
POST	/v1/auth/apple	none	Exchange Sign in with Apple identity token for backend session (JWT access + refresh)
POST	/v1/auth/refresh	refresh token	Rotate refresh token, issue new access token (~1hr lifetime)

Catalog

Method	Path	Auth	Description
GET	/v1/packs	none	List published packs (metadata + <code>apple_product_id</code> for client-side StoreKit lookup)
GET	/v1/packs/:slug	none	Pack detail, including field/filter framework

Entitlements & Purchases

Method	Path	Auth	Description
POST	/v1/purchases/verify	user	Client submits StoreKit transaction; server verifies via App Store Server API, creates/updates entitlement
GET	/v1/entitlements	user	List current user's owned packs (includes <code>revoked_at</code> state)
POST	/v1/entitlements/restore	user	Re-derive entitlements from Apple's transaction history (reinstall / new device)
POST	/v1/webhooks/app-store-notifications	Apple signature	App Store Server Notifications V2 — refund/revoke events set <code>entitlements.revoked_at</code>

Pack Delivery

Method	Path	Auth	Description
GET	/v1/packs/:slug/manifest	user	Latest version, <code>format_version</code> , checksum, POI count — used for update checks

GET	/v1/packs/:slug/download	user + entitlement check (revoked_at IS NULL)	Short-lived signed URL or streamed file, Range / Accept-Ranges / Content-Range supported for resumable downloads
-----	--------------------------	--	--

App Updates

Method	Path	Auth	Description
GET	/v1/app/version	none	Latest/minimum supported app version

Feedback

Method	Path	Auth	Description
POST	/v1/feedback	optional	Submit a data-correction / support note (pack, POI name/address, free-text message)

Admin (separate admin auth, not exposed to end-user app)

Method	Path	Description
POST	/v1/admin/poi-types	Create a POI type
POST	/v1/admin/packs	Create a pack (standard or state)
PUT	/v1/admin/packs/:id	Update pack metadata / status
POST	/v1/admin/packs/:id/framework/fields	Define a pack field
POST	/v1/admin/packs/:id/framework/filters	Define a pack filter
POST	/v1/admin/packs/:id/build	Rebuild pack_pois snapshot from master + cut a new pack_version
POST	/v1/admin/pois	Add a single POI to the master database
PUT	/v1/admin/pois/:id	Edit a master POI
POST	/v1/admin/pois/:id/geocode	Manually trigger/retry geocoding via Smarty
POST	/v1/admin/imports	Upload a dataset file, start an import job
GET	/v1/admin/imports/:id	Import status + row-level errors
GET	/v1/admin/feedback	List/triage feedback submissions

8. Use Cases

End user

UC-1 — Search a route

- **Actor:** End user
- **Precondition:** At least one pack is installed and active.
- **Flow:**
 - i. User opens the route search screen.
 - ii. User enters/selects a start location.
 - iii. User enters/selects an end location.
 - iv. User selects a search radius (1, 5, 10, 25, or 50 miles).
 - v. App calculates the route via MapKit.
 - vi. App samples the route (§10) and queries on-device pack data for active packs within the selected radius.
 - vii. App displays the result list, with each POI attributed to its originating pack.

UC-2 — View POI detail

- **Actor:** End user
- **Precondition:** A route search has returned at least one POI.
- **Flow:**
 - i. User taps a POI in the results list.
 - ii. App opens the POI detail view.
 - iii. App displays name, full address, phone (if present), website (if present), and pack-specific filter values.
 - iv. App displays the `additional_info` note only if it is non-empty.
 - v. User navigates back to the results list.

UC-3 — Purchase a pack

- **Actor:** End user
- **Precondition:** User is signed in with Apple; the pack is listed in the catalog and not already owned.
- **Flow:**
 - i. User browses the Pack Store and selects a pack.
 - ii. User taps "Buy."
 - iii. App initiates a StoreKit purchase and the user completes the Apple payment sheet.
 - iv. App finishes the StoreKit transaction locally and sends it to `/v1/purchases/verify`.
 - v. Backend verifies the transaction via the App Store Server API and records the entitlement.
 - vi. App begins downloading the pack, independently of transaction completion (a failed download does not affect the entitlement — see UC-8).
 - vii. Pack installs locally and auto-activates (§12).

UC-4 — Restore purchases

- **Actor:** End user

- **Precondition:** User is signed in with Apple on a new or reinstalled device; previously purchased packs are not present locally.
- **Flow:**
 - i. User taps "Restore Purchases."
 - ii. App calls `/v1/entitlements/restore`.
 - iii. Backend queries the App Store Server API for the account's transaction history and reconciles the `entitlements` table.
 - iv. App fetches the current entitlement list via `/v1/entitlements`.
 - v. App downloads each owned pack not already present locally.

UC-5 — Filter within a pack

- **Actor:** End user
- **Precondition:** At least one active pack has filter definitions; a route search has been performed.
- **Flow:**
 - i. User opens the filters panel for an active pack.
 - ii. User selects one or more filter values.
 - iii. App re-evaluates the current result list against `filter_values` matching the selection.
 - iv. App updates the displayed list to matching POIs only.

UC-6 — Manage active packs

- **Actor:** End user
- **Precondition:** Two or more packs are installed.
- **Flow:**
 - i. User opens Pack Management.
 - ii. App lists installed packs with an active/inactive toggle.
 - iii. User toggles a pack off.
 - iv. App deactivates the pack, removes its POIs from the current result list, and grays out (but does not hide) its filters.
 - v. User toggles the pack back on; its POIs and filters return to normal.

UC-7 — Buy and use a State Pack

- **Actor:** End user
- **Precondition:** A state pack is available for purchase and not yet owned.
- **Flow:**
 - i. User purchases the state pack (follows UC-3).
 - ii. Pack installs and auto-activates.
 - iii. User performs a route search that crosses the pack's state and other states.
 - iv. App returns only state-pack POIs within the radius of the route — in practice, only where the route passes through that state.

UC-8 — Receive updates

- **Actor:** End user

- **Precondition:** Device is online; a newer app version or pack version exists.
- **Flow:**
 - i. App launches (or the user manually triggers a check).
 - ii. App calls `/v1/app/version` and, for each installed pack, `/v1/packs/:slug/manifest`.
 - iii. App compares versions and `format_version` against local state.
 - iv. If a pack update is available, app downloads the new `pack_version` (resumable) and validates its checksum before swapping it in.
 - v. If the app itself is below the minimum supported version, app shows an update-required state.

UC-9 — Handle a revoked pack

- **Actor:** End user (triggered indirectly via Apple's refund flow)
- **Precondition:** A previously purchased pack's entitlement has been revoked/refunded via Apple.
- **Flow:**
 - i. Apple sends a refund/revoke notification to `/v1/webhooks/app-store-notifications`.
 - ii. Backend sets `entitlements.revoked_at`.
 - iii. On the next entitlement check (launch, restore, or download attempt), app/backend detects `revoked_at IS NOT NULL`.
 - iv. App blocks further downloads/updates for that pack and surfaces `ENTITLEMENT_REVOKED` messaging if the user attempts one.
 - v. Already-downloaded local pack data remains on-device and searchable; the app does not force-delete it.

UC-10 — Submit feedback

- **Actor:** End user
- **Precondition:** None — accessible from a POI detail view (pre-filled) or Settings (blank); sign-in not required.
- **Flow:**
 - i. User taps "Report an issue" from a POI detail view or Settings.
 - ii. User enters a free-text message.
 - iii. User submits the form.
 - iv. App calls `POST /v1/feedback`.
 - v. Backend stores the submission with `status = 'open'`.
 - vi. App shows a confirmation to the user.

Admin (product owner)

UC-11 — Author a pack framework

- **Actor:** Admin
- **Precondition:** Admin is authenticated in the web admin app; a pack record exists (or is being created).
- **Flow:**
 - i. Admin creates or opens a pack in `draft` status.
 - ii. Admin defines fields (key, label, data type, required flag) via `/v1/admin/packs/:id/framework/fields`.

- iii. Admin defines filters (key, label, type, options) via `/v1/admin/packs/:id/framework/filters` .
- iv. Admin reviews the framework summary before proceeding to data population.

UC-12 — Add a POI manually

- **Actor:** Admin
- **Precondition:** Target POI type exists; admin is authenticated.
- **Flow:**
 - i. Admin opens "Add POI."
 - ii. Admin enters name, address fields, optional phone/website, `additional_info` , and any pack-specific custom field/filter values.
 - iii. Admin submits via `POST /v1/admin/pois` .
 - iv. Backend geocodes the address via Smarty and stores lat/lon, `geocode_status` , and `geocode_confidence` .
 - v. If geocoding fails or is low-confidence, the POI is saved but flagged for review and excluded from pack builds.

UC-13 — Import a dataset

- **Actor:** Admin
- **Precondition:** A dataset file exists for a target POI type/pack.
- **Flow:**
 - i. Admin uploads the file via `POST /v1/admin/imports` , selecting the target POI type.
 - ii. Backend validates the file contains all columns required by the target pack's framework; if not, the import halts with `MISSING_REQUIRED_COLUMN` .
 - iii. Backend processes rows: valid rows are normalized, geocoded via Smarty, and inserted/updated in the master `pois` table.
 - iv. Rows with invalid/missing optional data are flagged (`INVALID_FORMAT` / `MISSING_OPTIONAL_FIELD`) but the import continues.
 - v. Rows matching an existing master POI (normalized name + street + city/state) are flagged `DUPLICATE_SUSPECTED` , not auto-merged.
 - vi. Backend marks the import `completed` , with total/flagged row counts.
 - vii. Admin reviews flagged rows via `GET /v1/admin/imports/:id` and corrects them manually.

UC-14 — Build/rebuild a standard pack

- **Actor:** Admin
- **Precondition:** Pack framework is defined; the master DB contains POIs of the pack's POI type with `geocode_status = 'ok'` .
- **Flow:**
 - i. Admin triggers `POST /v1/admin/packs/:id/build` .
 - ii. Backend selects all active, successfully-geocoded master POIs of the pack's `poi_type_id` .
 - iii. Backend validates every required framework field is present on each selected POI; if any are missing, the build fails and reports which POIs/fields are missing.
 - iv. Backend regenerates the `pack_pois` snapshot.
 - v. Backend generates a pack file, computes its checksum, and creates a new `pack_versions` row.

- vi. Admin publishes the pack (or new version) once satisfied.

UC-15 — Build/rebuild a state pack

- **Actor:** Admin
- **Precondition:** A state pack record exists with `state_code` set; the master DB contains geocoded POIs of any type in that state.
- **Flow:**
 - i. Admin triggers `POST /v1/admin/packs/:id/build` for the state pack.
 - ii. Backend selects all active, successfully-geocoded master POIs where `address_state = state_code`, across all POI types.
 - iii. Backend regenerates the `pack_pois` snapshot, preserving each POI's originating pack-of-origin filters/fields.
 - iv. Backend generates a new pack file/version as in UC-14.

UC-16 — Triage feedback

- **Actor:** Admin
- **Precondition:** One or more `open` feedback submissions exist.
- **Flow:**
 - i. Admin opens the feedback queue via `GET /v1/admin/feedback`.
 - ii. Admin reviews a submission's pack/POI reference and message.
 - iii. Admin makes the corresponding correction (edit POI, re-geocode, etc.) via the relevant admin tool.
 - iv. Admin marks the submission `resolved`.

9. Test Cases

Route search & filtering (UC-1, UC-5, UC-6, UC-7)

TC-1

- **Given:** two packs are installed and active, each with POIs within range of a route
- **When:** the user searches the route
- **Then:** the result list shows POIs from both packs, each correctly attributed to its originating pack

TC-2

- **Given:** a completed route search
- **When:** the user changes the radius from 1 to 50 miles
- **Then:** results update to the new radius without triggering a new route calculation

TC-3

- **Given:** only a Colorado state pack is active
- **When:** the user searches a route from Denver, CO to Dallas, TX
- **Then:** only POIs within the radius of the Colorado portion of the route are shown

TC-4

- **Given:** a pack is active with POIs shown in the result list
- **When:** the user deactivates that pack
- **Then:** its POIs disappear from the results, and its filters remain visible but grayed out, not hidden

TC-5

- **Given:** active packs include one with a "dog-friendly" filter
- **When:** the user selects the "dog-friendly" filter
- **Then:** the result list narrows to matching POIs from currently active packs only

Empty / low-result states (Section 11)

TC-6

- **Given:** the user has no packs installed
- **When:** the user opens route search
- **Then:** the app shows the "no packs" empty state with a Browse Store CTA, not a search error

TC-7

- **Given:** packs are installed but none are active
- **When:** the user performs a route search
- **Then:** the app shows "no active packs" with a shortcut to pack management

TC-8

- **Given:** active packs exist but no POIs fall within the selected radius
- **When:** the user performs a route search
- **Then:** the app shows "no stops within {radius}" with a one-tap radius bump action

TC-9

- **Given:** a route search returns POIs but the user's active filters exclude all of them
- **When:** filters are applied
- **Then:** the app shows "no matches for your filters," distinct from the zero-POI state, with a clear-filters action

Route sampling (Section 10)

TC-10

- **Given:** a short in-town route
- **When:** the route is sampled
- **Then:** the exact start and end coordinates are included as samples regardless of interval

TC-11

- **Given:** a multi-thousand-mile cross-country route and a small search radius

- **When:** the route is sampled
- **Then:** the interval widens to respect the 500-sample soft cap while keeping the max gap between samples $\leq 2 \times$ the search radius

TC-12

- **Given:** an extremely long route combined with a very small radius
- **When:** honoring the max-gap invariant would exceed the hard ceiling of 2000 samples
- **Then:** the app accepts degraded coverage and logs it prominently rather than silently proceeding

Purchases & entitlements (UC-3, UC-4, UC-9)

TC-13

- **Given:** a signed-in user completes a StoreKit purchase for an unowned pack
- **When:** the backend verifies the transaction with the App Store Server API
- **Then:** an entitlement is recorded and the pack becomes downloadable

TC-14

- **Given:** a user initiates a purchase
- **When:** the StoreKit transaction fails or is cancelled
- **Then:** no entitlement is created, the user sees a clear error, and no partial download occurs

TC-15

- **Given:** a user already owns a pack
- **When:** the user attempts to "buy" it again
- **Then:** the app recognizes the existing entitlement and offers a download instead of a duplicate charge

TC-16

- **Given:** a user previously purchased packs and reinstalls the app
- **When:** the user taps Restore Purchases
- **Then:** all previously purchased packs reappear as owned and become re-downloadable

TC-17

- **Given:** a user does not own a given pack
- **When:** the client calls `GET /v1/packs/:slug/download`
- **Then:** the request is rejected with 403 and no file is served

TC-18

- **Given:** an owned pack's purchase is refunded
- **When:** Apple sends a revoke notification
- **Then:** `revoked_at` is set, future downloads/updates are blocked, and already-downloaded local pack data is left untouched

TC-19

- **Given:** a purchase is made in the App Store sandbox
- **When:** the transaction is verified
- **Then:** the entitlement is recorded with `environment = 'sandbox'` and excluded from production conversion reporting

Pack delivery & updates (UC-8)

TC-20

- **Given:** an installed pack has a newer version published on the backend
- **When:** the app checks the pack's manifest
- **Then:** the app detects the version mismatch and prompts/auto-downloads the update

TC-21

- **Given:** a pack download completes
- **When:** the app verifies the downloaded file's checksum
- **Then:** a mismatch triggers an automatic retry rather than installing corrupted data

TC-22

- **Given:** a pack download is interrupted, e.g. the app is backgrounded
- **When:** the download resumes
- **Then:** it continues from the last byte offset via `Range` rather than restarting from zero

TC-23

- **Given:** a pack's `format_version` is newer than the app supports
- **When:** the app attempts to install it
- **Then:** installation is refused, and the app prompts for an app update rather than attempting a partial parse

TC-24

- **Given:** the installed app version is below the minimum supported version
- **When:** the app checks `/v1/app/version`
- **Then:** the app surfaces an update-required state

On-device schema migration

TC-25

- **Given:** a pending on-device schema migration exists
- **When:** the app launches
- **Then:** the migration runs to completion before any pack install/update path becomes available

TC-26

- **Given:** a schema migration fails

- **When:** the app attempts to proceed
- **Then:** pack operations are blocked and an explicit "update required / contact support" state is shown, with no partial or corrupted local state

Admin: framework & pack building (UC-11, UC-14, UC-15)

TC-27

- **Given:** a pack framework defines a required field
- **When:** the admin builds the pack and at least one included POI is missing that field
- **Then:** the build fails and reports which POIs/fields are missing

TC-28

- **Given:** a new POI of a pack's POI type is added to the master DB
- **When:** the admin rebuilds the standard pack
- **Then:** the new POI appears in the resulting `pack_version`

TC-29

- **Given:** a state pack covering multiple POI types
- **When:** the admin builds the pack
- **Then:** it includes POIs of all types located in that state, each retaining its own pack-of-origin filters

Admin: import pipeline (UC-13)

TC-30

- **Given:** an import file is missing a column required by the target pack's framework
- **When:** the import runs
- **Then:** it halts immediately with `MISSING_REQUIRED_COLUMN`, naming the missing field

TC-31

- **Given:** an import row has an invalid or missing optional value, e.g. a malformed phone number
- **When:** the import processes that row
- **Then:** the row is flagged (`INVALID_FORMAT` / `MISSING_OPTIONAL_FIELD`) and the rest of the import proceeds

TC-32

- **Given:** an import row has an unresolvable address
- **When:** geocoding is attempted
- **Then:** `geocode_status` is set to `failed`, the POI is saved, and it is excluded from pack builds until corrected

TC-33

- **Given:** an import row matches an existing master POI by normalized name + street + city/state

- **When:** the row is processed
- **Then:** it is flagged `DUPLICATE_SUSPECTED` and never auto-merged

TC-34

- **Given:** a fully valid import file
- **When:** the import completes
- **Then:** all rows are geocoded, added to the master DB, and available for the next pack build

10. Route Sampling Algorithm

Determines which points along a MapKit route are used to evaluate "is this POI within the selected radius of the route." Lives in a single shared module, unit-tested against both short in-town routes and multi-thousand-mile cross-country routes.

1. Always include the exact start and end coordinates as samples, regardless of interval.
2. Target interval = `min(radius_miles, 10)` miles — the interval never exceeds the search radius, which is what guarantees no gap in the corridor can hide a POI in the un-widened case.
3. Soft cap: 500 samples. If `route_length_miles / interval` would exceed the cap, widen the interval to fit — but never let the widened interval push the max gap between consecutive samples past **2× the search radius** (the correctness invariant).
4. If honoring that invariant would still require exceeding the soft cap, raise the sample count up to a hard ceiling (2000) rather than silently accepting worse coverage.
5. If even the hard ceiling isn't enough (an extremely long route paired with a very small radius), accept degraded coverage and log it prominently — this should be a rare edge case, not a normal path.

Spatial search (per-search execution)

1. From the sample points (not a single midpoint), compute `min/max latitude` and `min/max longitude`, then pad by the selected radius (converted to degrees) — this bounds the query correctly on long diagonal routes instead of over- or under-sizing the candidate box.
2. Query the local `pois` table (indexed on `(latitude, longitude)`) for candidates inside that bounding box, scoped to active packs.
3. Run haversine distance from each candidate to its nearest route sample; keep results within the selected radius.
4. **Phase 1 exit criteria:** benchmark this pipeline with a synthetic 10,000-POI pack, and again with 2-3 packs active simultaneously (time + memory), on a representative lower-end device. Use the results to set the "many large packs active" soft-warning threshold (§12) and to decide whether the bounding-box pre-filter is sufficient or an R-tree index is warranted.

11. Empty / Low-Result States

Four distinct cases, driven by a single pure function of `(installed_packs, active_packs, raw_result_count,`

filtered result count) so client UI stays consistent:

Case	Condition	Message	Primary action
No packs	Zero packs installed	"You don't have any packs yet"	Browse the Pack Store
No active packs	Packs installed, none active	"Turn on a pack to see results"	Go to pack management
No POIs in radius	Active packs, raw result count = 0	"No stops found within {radius} mi"	Bump to next radius tier
Filtered to zero	Raw results > 0, filtered result count = 0	"No matches for your filters"	Clear filters

Copy stays short; primary action must be equally obvious on iPhone and iPad layouts.

12. Pack Activation Defaults

- Newly purchased/downloaded packs **auto-activate** — least surprising default.
- No hard cap on simultaneously active packs.
- Past a POI-count threshold (set after the Phase 1 synthetic benchmark, §10), show a soft "search may be slower with this many packs active" indicator rather than blocking anything.

13. Versioning & Migration Strategy

Two independent version numbers:

- **On-device schema version** — tracked via SQLite `PRAGMA user_version` ; ordered migration scripts run on app launch, before any pack install/update path is allowed to proceed. A failed migration blocks pack operations and surfaces an explicit "update required / contact support" state rather than failing silently or partially.
- **Pack file format version** — carried in the pack manifest (`pack_versions.format_version`). The app refuses to install a pack file format newer than it understands and prompts an app update rather than attempting a partial parse. Format version bumps should be rare; most data changes stay within the same format.

14. Analytics (privacy-respecting)

- First-party only — own backend, no third-party SDK.
- Keyed to a random install UUID, not the Apple account identifier (keeps it out of "linked to identity" on the privacy label).
- Settings opt-out toggle.

- Batched, periodic flush (foreground/background transitions), dropped entirely if the user opts out.
- Minimal event set, no coordinates or precise identifiers:
 - `search_performed` (radius, active_pack_count, result_count)
 - `pack_purchased` (pack_id)
 - `pack_activated` / `pack_deactivated` (pack_id)
 - `filter_applied` (pack_id, filter_key)
 - `pack_update_downloaded` (pack_id, version)

15. Error Taxonomy

Each error carries a machine code, a user/admin-facing message, and a suggested action.

Download (client-facing)

Code	Message	Action
<code>NETWORK_UNAVAILABLE</code>	"Check your connection and try again"	Retry button
<code>CHECKSUM_MISMATCH</code>	"Download was corrupted — retrying"	Auto-retry once, then manual
<code>ENTITLEMENT_NOT_FOUND</code> (403)	"This pack isn't linked to your account"	Prompt Restore Purchases
<code>ENTITLEMENT_REVOKED</code>	"This pack is no longer available on your account"	No retry — distinct from <code>ENTITLEMENT_NOT_FOUND</code> so the client doesn't mislead the user into restoring
<code>STORAGE_FULL</code>	"Not enough space to download this pack"	—
<code>SERVER_UNAVAILABLE</code> (5xx)	"Trouble on our end — try again shortly"	Retry with backoff

Geocoding (admin-facing, per POI)

Code	Meaning	Action
<code>ADDRESS_NOT_FOUND</code>	Provider found zero matches	Manual correction needed
<code>ADDRESS_AMBIGUOUS</code>	Multiple plausible matches	Candidates stored in <code>geocode_candidates</code> ; admin selects
<code>LOW_CONFIDENCE_MATCH</code>	Match below confidence threshold	Flag for review, excluded from pack builds until resolved
	Rate-limited or	Auto-retry with backoff; surface to admin only once

PROVIDER_ERROR	transient failure	retries are exhausted
----------------	-------------------	-----------------------

Import (admin-facing)

Code	Level	Behavior
MISSING_REQUIRED_COLUMN	File	Hard stop
UNREADABLE_FILE	File	Hard stop
MISSING_OPTIONAL_FIELD	Row	Flag, continue
INVALID_FORMAT	Row	Flag, continue
GEOCODE_FAILED	Row	Flag, continue
DUPLICATE_SUSPECTED	Row	Flag, continue — never auto-merge, admin stays in control

Duplicate matching rule: normalized name + street + city/state (not pure coordinate proximity, which produces false positives for e.g. adjacent businesses in the same plaza).

16. Auth & Offline Token Policy

- Access tokens: ~1 hour lifetime — long enough that short offline stretches don't force a refresh.
- Refresh tokens: stored in `expo-secure-store`, rotated on use.
- Silent refresh is attempted on launch but **never blocks** the core offline path — route search against already-downloaded packs must work with zero network and zero valid session.
- Forced re-login is triggered only when the user attempts a network-requiring action (purchase, download, restore, entitlement check) and the refresh token is dead — never simply because the app was opened offline.

17. Geocoding Strategy

- Provider: **Smarty** (US address standardization + geocoding in a single call; terms permit permanent storage of results, unlike Google's Geocoding API).
- Integrate behind a thin adapter so the provider can be swapped later without touching import/admin logic.
- Cache by a normalized address key (street + city + state + ZIP, lowercased/stripped) so re-imports or minor data refreshes don't re-bill.
- Store a confidence score (`geocode_confidence`) and route below-threshold matches to `low_confidence` status — excluded from pack builds until manually reviewed.
- Ambiguous matches store candidates (`geocode_candidates`) for admin selection rather than auto-picking the top result.
- **Pilot import:** before committing to a full seed dataset, run 50-100 real addresses from the intended launch pack through the full pipeline (normalize → geocode → store → build a pack file → install on

device) to surface address-quality or confidence-threshold issues early. Scheduled as a Phase 1 task (§18).

18. Development Phases

Phase	Goal	Key deliverables
0 — Foundations	Prove the core mechanic end-to-end	VPS service scaffold (<code>wowm-api</code> + <code>wowm-db</code> , nginx site); Apple Developer setup (App ID, Sign in with Apple, StoreKit sandbox); iOS app shell (Expo); MapKit route + radius search against a small hardcoded POI set — no packs, no IAP, no location permission
1 — Single Pack MVP	One real pack, working offline search, validated performance	Smarty adapter + pilot import (50-100 addresses, §17); seed first real pack in master DB; authenticated (non-paywalled) download; on-device SQLite storage; bounding-box + haversine local search; composite lat/lon indexes; synthetic 10k-POI benchmark + multi-pack (2-3 active) benchmark as exit criteria; pack-specific filters UI; POI detail view; four empty states
2 — Commerce	Real purchases	StoreKit product per pack; purchase flow decoupled from download; server-side receipt verification via App Store Server API; App Store Server Notifications V2 webhook (refund/revoke → <code>revoked_at</code>); Family Sharing enabled; entitlements; restore purchases; <code>ENTITLEMENT_REVOKED</code> handling
3 — Multi-Pack	Support owning/using several packs	Second pack purchasable; pack-selection UI; filter area grouped by pack with inactive packs grayed out; pack auto-activation + soft performance warning; app/pack update-check with resumable downloads and format-version guard
4 — State Packs & Pack Tooling	Admin can author and build packs	<code>poi_types</code> ; pack framework builder (fields/filters) in admin web app; pack build/rebuild tool; first state pack
5 — Import Pipeline	Scale data entry beyond manual add	Admin file import UI; validation (flag invalid/missing, hard-stop on missing required field, duplicate detection); geocoding step; import status/error reporting; in-app feedback channel (UC-10, UC-16)
6 — Polish & Launch	Ready for App Store	App Store submission assets (screenshots, privacy nutrition label — minimal, given no location permission); on-device schema migration hardening; VPS monitoring/backup coverage extended to <code>wowm-db</code> ; revisit R-tree spatial indexing only if real-pack-size data shows the bounding-box approach insufficient

Open items for future discussion

- Exact UI/UX flows (wireframes) for the three critical screens (route search results, pack management + filters, POI detail) — sequenced right after the minimum launch pack set is chosen, since real field/filter shape should drive the design rather than placeholders.
- Minimum viable launch pack set and pricing tiers — one strong vertical (high name recognition, dense enough locations) plus, bandwidth permitting, one complementary pack so multi-pack behavior gets exercised early.
- Whether admin auth is a separate, simpler mechanism (e.g. single shared login) given single-admin usage in early phases.
- Future "use current location" convenience button — deferred by design (\$1); revisit once usage data justifies the added permission/review overhead.